

Project Documentation: Car Game

Shoaib Rafiq

Fall-23-BSCS-479

Abdullah

Fall-23-BSCS-478

Usama

Fall-23-BSCS-476

Rafiqmayo0987@gmail.com

Abstract

The Car Game is a dynamic arcade-style game where players control a car to navigate through obstacles on a highway. The game combines engaging gameplay mechanics, responsive controls, and a scoring system that tracks player performance. This document outlines the game's objectives, algorithmic foundations, rules, design elements, testing strategies, and areas for optimization. The focus is on creating an interactive and enjoyable user experience while ensuring efficient algorithm implementation.

Keywords

- Collision Detection
- Arcade Gameplay
- Obstacle Avoidance
- Scoring System
- Leaderboard Integration

Introduction

The Car Game is inspired by classic arcade racing games that emphasize reflexes and precision. Players navigate a car through lanes while avoiding enemy vehicles. The objective is to achieve the highest score by traveling the greatest distance and avoiding collisions. This document explores the game's mechanics, problem definition, algorithmic design, and implementation details.

About the Game

Objective

Players aim to navigate their car through enemy vehicles while avoiding collisions. The longer the car stays on the road without crashing, the higher the score.

Rules

1. Players control the car using keyboard keys (A for left, D for right).
2. Colliding with an enemy vehicle ends the game.
3. A scoring system rewards players based on the distance traveled.
4. The leaderboard stores and displays top scores.
5. Game includes visual boundaries to restrict movement.

Game Elements

1. **Player Car:** A 4x4 grid-controlled car displayed on the lower section of the screen.
2. **Enemy Vehicles:** Obstacle cars generated randomly in the lanes.
3. **Score Display:** Tracks and updates the player's score in real time.
4. **Boundaries:** Visual borders to define the playable area.

Initial State

- Player car is positioned in the center of the playable area.
- Enemy vehicles are randomly placed at the top of the screen.
- Score is initialized to zero.

Objective State

- Achieve the highest score possible without colliding with enemy vehicles.
- Avoid collisions while maintaining control of the car.

Actions

1. **Move Left:** Press 'A' key.
2. **Move Right:** Press 'D' key.
3. **Quit Game:** Press 'ESC' key.

Move Validation

- Left movement is valid if the car remains within the left boundary.
- Right movement is valid if the car remains within the right boundary.

Transition Function

Each valid player action updates the car's position. Collisions trigger the game over state.

Game Loop

1. Detect player input for car movement.
2. Update positions of the player car and enemy vehicles.
3. Check for collisions or boundary violations.
4. Update the score based on elapsed time.
5. Repeat until the game ends.

Algorithm Design

Problem Statement

Design an efficient algorithm to manage car movement, enemy vehicle generation, collision detection, and scoring, ensuring real-time responsiveness and smooth gameplay.

Algorithm Objectives

1. Efficiently detect collisions between player car and enemy vehicles.
2. Randomly generate enemy vehicles within defined lanes.
3. Implement a scoring system based on player survival time.

4. Save scores to a file and display the leaderboard.

Proposed Solution

The game leverages:

- **Randomization:** For generating enemy vehicles in random lanes.
- **Collision Detection:** To check overlaps between the player car and enemy vehicles.
- **File I/O:** To save and retrieve scores for the leaderboard.

Key Features

1. **Car Movement:** Updates player car's position based on input.
 2. **Enemy Generation:** Randomly places enemy vehicles in lanes.
 3. **Collision Check:** Ends the game when the player collides with an enemy vehicle.
 4. **Score Tracking:** Increments the score for every successful frame.
 5. **Leaderboard:** Displays top scores from saved files.
-

Testing and Iteration

Testing Plan

1. **Unit Testing:** Validate car movement, enemy generation, and collision detection.
2. **Integration Testing:** Ensure all components interact correctly.
3. **Edge Cases:**
 - Multiple simultaneous collisions.
 - Player car at extreme boundaries.
 - High-speed gameplay.

Iteration Process

1. Collect feedback from user testing.
 2. Refine algorithms based on identified issues.
 3. Optimize collision detection and randomization logic.
-

Optimization

1. **Collision Detection:** Implement bounding box checks to reduce computational overhead.
 2. **Enemy Movement:** Use efficient data structures for managing enemy positions.
 3. **Scalability:** Ensure smooth performance even with multiple enemy vehicles on screen.
 4. **Visual Enhancements:** Add animations for smoother gameplay.
-

Conclusion

The Car Game provides an engaging arcade experience with real-time controls and scoring. Algorithmic optimizations ensure smooth gameplay, and the leaderboard fosters competitiveness. Future improvements include enhanced graphics, power-ups, and additional game modes.
